

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12
SOLID PRINCIPLES



Disusun Oleh:
Iman Dwi Satrio 105224029

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA
2026

I. Pendahuluan

Praktikum Pemrograman Berorientasi Objek (PBO) pada modul ini berfokus pada pemahaman dan implementasi lima prinsip desain SOLID (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, dan Dependency Inversion) di dalam pemrograman Java. Prinsip SOLID merupakan fondasi dalam rekayasa perangkat lunak guna mentransformasikan kode yang kaku, rapuh, dan sulit diubah menjadi arsitektur aplikasi yang modular, tangkas, serta adaptif terhadap perluasan fitur. Penerapan SOLID memisahkan setiap domain tanggung jawab secara terisolasi melalui pemanfaatan abstraksi antarmuka.

Implementasi studi kasus pada laporan praktikum ini diterapkan pada perancangan ulang sistem portal akademik SIAKAD kampus yang mengalami kegagalan sistem akibat penumpukan beban logika operasional. Sebelum rekonstruksi dilakukan, sistem bertumpu sepenuhnya pada kelas SistemKRSManager yang mengurus validasi, kalkulasi UKT, pembuatan draf PDF, hingga koneksi database. Melalui integrasi berlapis dari kelima prinsip SOLID, sistem SIAKAD direkayasa ulang guna menjamin stabilitas aplikasi saat menghadapi beban akses tinggi (war KRS) serta memisahkan ketergantungan keras logika bisnis dari infrastruktur teknis database.

Prinsip Single Responsibility Principle (SRP) diterapkan untuk memecah tanggung jawab SistemKRSManager ke dalam komponen validator akademik dan pencetak draf independen. Evaluasi skema perhitungan UKT yang dipenuhi ratusan baris pengkondisian (if-else) direkonstruksi menggunakan Open/Closed Principle (OCP) berbasis Strategy Pattern agar siap menerima perluasan skema program "Merdeka Belajar Kampus Merdeka" (MBKM) tanpa memodifikasi rumus lama. Penanganan jenis mata kuliah (Teori dan Praktikum) diselesaikan dengan Liskov Substitution Principle (LSP) dan Interface Segregation Principle (ISP) untuk mengeliminasi polusi metode dan mencegah interupsi runtime error berupa UnsupportedOperationException. Terakhir, arsitektur MySQL diputus menggunakan Dependency Inversion Principle (DIP) melalui pembuatan abstraksi interface koneksi, sehingga mempermudah proses migrasi ke infrastruktur Cloud NoSQL.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	db	Database Connection	Menyimpan referensi objek database berbasis abstraksi interface (DIP), menggantikan referensi konkret MySQLDatabaseConnection terdahulu.
2	nama	String	Menyimpan informasi nama mata kuliah yang bersifat teori untuk keperluan registrasi kelas.

3	nama	String	Menyimpan informasi nama mata kuliah praktikum untuk keperluan registrasi dan operasional laboratorium.
---	------	--------	---

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	SistemKRSManager(DatabaseConnection db)	Constructor	Berperan sebagai wadah Constructor Injection untuk menyuntikkan objek database dari luar secara dinamis (DIP).
2	kalkulasiUKT (KalkulasiJalurUKT jalurMasuk)	Functional	Mendelegasikan proses perhitungan UKT secara polimorfis tanpa struktur if-else berdasarkan objek strategi yang dikirim (OCP).
3	simpanKRSKedatabase(String mahasiswa, String matakuliah)	Procedural	Memerintah objek abstraksi db untuk membuka koneksi dan mengamankan riwayat pengambilan KRS ke media penyimpanan.
4	hitung()	Functional	Kontrak metode polimorfisme (interface KalkulasiJalurUKT) yang wajib diimplementasikan oleh setiap variasi jalur masuk mahasiswa (Reguler, Karyawan, MBKM, dll).
5	validasiPrasyaratMataKuliah (String mahasiswa, String matakuliah)	Functional	Melakukan pengecekan kelayakan akademik mahasiswa sebelum diizinkan mengambil suatu kelas (SRP).
6	cetakDraftKRS(String mahasiswa, String matakuliah)	Procedural	Menangani seluruh urusan visual presentasi dan pembuatan dokumen draf KRS dalam format PDF (SRP).
7	registrasiKelas()	Procedural	Kontrak dasar pendaftaran kelas yang dijamin aman untuk dieksekusi oleh seluruh jenis

			mata kuliah tanpa memicu crash (LSP).
8	alokasiAsisten Lab()	Procedural	Kontrak spesifik untuk menugaskan asisten laboratorium pada mata kuliah yang memiliki fasilitas praktikum (ISP).
9	cekPeralatanP raktikum()	Procedural	Kontrak spesifik untuk melakukan pengecekan inventaris alat lab sebelum praktikum dimulai (ISP).
10	hubungkan()	Procedural	Kontrak abstraksi tingkat rendah untuk menginisialisasi jembatan komunikasi ke server database (DIP).
11	simpanKRS(S tring mahasiswa, String matakuliah)	Functional	Kontrak abstraksi tingkat rendah untuk melakukan operasi insert data KRS mahasiswa ke penyimpanan (DIP).

IV. Dokumentasi dan Pembahasan Code

Bagian ini akan membahas logika pemrograman awal yang belum SOLID.

```

1 + class SistemKRSManager {
2 +     private MySQLDatabaseConnection db; // Ketergantungan langsung (Hardcoded)
3 +
4 +     public SistemKRSManager() {
5 +         this.db = new MySQLDatabaseConnection(); // menginstansiasi secara langsung objek MySQLDatabaseConnection
6 +     }
7 +
8 +     // Fungsi Validasi Prasyarat
9 +     public boolean validasiPrasyaratMatakuliah(String mahasiswa, String matakuliah) {
10 +         System.out.println("Memvalidasi prasyarat akademik untuk " + mahasiswa);
11 +         return true;
12 +     }
13 +
14 +     // If-Else untuk perhitungan UKT berdasarkan Jalur Masuk
15 +     public double hitungUangKuliahTunggal(String jalurMasuk) {
16 +         if (jalurMasuk.equalsIgnoreCase("Reguler")) {
17 +             return 5000000;
18 +         } else if (jalurMasuk.equalsIgnoreCase("Karyawan")) {
19 +             return 7500000;
20 +         } else if (jalurMasuk.equalsIgnoreCase("Internasional")) {
21 +             return 15000000;
22 +         } else if (jalurMasuk.equalsIgnoreCase("Bidikmisi")) {
23 +             return 0;
24 +         } else {
25 +             return 3000000;
26 +         }
27 +     }
28 +
29 +     // Fungsi Cetak PDF yang bercampur dengan logika penyimpanan database
30 +     public void cetakDraftKRS(String mahasiswa, String matakuliah) {
31 +         System.out.println("Proses Cetak...");
32 +         // Pada semester lalu, seorang pemrogram mengubah format kop surat pada file PDF KRS,
33 +         // Simulasi perubahan format string/kop memicu interupsi/bug
34 +         boolean adaKesalahanFormatKop = true;
35 +         if (adaKesalahanFormatKop) {
36 +             System.out.println("Format Kop Surat Rusak! Proses dihentikan.");
37 +             // Akibatnya baris kode penyimpanan di bawah terlewat/gagal dieksekusi
38 +             return;
39 +         }
40 +         // Algoritma penyimpanan data sehingga ribuan data KRS mahasiswa gagal tersimpan ke database.
41 +         db.hubungkan();
42 +         db.simpanKRS(mahasiswa, matakuliah);
43 +     }
44 + }

```

Penjelasan: Kelas SistemKRSManager bertindak sebagai kelas terpusat karena memikul lebih dari satu tanggung jawab operasional. Kelas ini mencampuradukkan logika bisnis validasi akademik, kalkulasi keuangan (UKT), pembuatan dokumen presentasi (cetak draf PDF), hingga manajemen koneksi database tingkat rendah. Ketiadaan batasan tanggung jawab yang jelas memicu interdependensi tinggi. Hal ini terbukti pada metode cetakDraftKRS(), di mana kegagalan sepele pada format tampilan draf PDF (adaKesalahanFormatKop = true) secara langsung memotong alur program (return) dan menggagalkan eksekusi penyimpanan data ke database. Perubahan pada satu aspek presentasi merusak stabilitas aspek persistensi data.

Selain itu, Metode hitungUangKuliahTunggal() melanggar OCP karena mengandalkan struktur seleksi kondisi if-else bertumpuk (If-Else Hell) untuk menentukan nominal UKT berdasarkan parameter teks hardcoded. Arsitektur ini bersifat kaku (rigid) dan rapuh (fragile). Ketika pihak Rektorat menuntut penambahan skema perhitungan SKS baru untuk program "Merdeka Belajar Kampus Merdeka" (MBKM), pengembang terpaksa harus melakukan modifikasi langsung pada tubuh metode ini dengan menyelipkan blok else if baru. Aktivitas membongkar kelas inti yang sudah stabil sangat berisiko merusak rumus kalkulasi jalur masuk lama (seperti Reguler atau Karyawan) akibat kesalahan manusiawi (human error).

```
TakeHomeTask/src/OperasiMataKuliah.java
... @@ -0,0 +1,5 @@
1 + interface OperasiMataKuliah {
2 +     void registrasiKelas();
3 +     void alokasiAsistenLab();
4 +     void cekPeralatanPraktikum();
5 + }

TakeHomeTask/src/MataKuliahTeori.java
... @@ -0,0 +1,16 @@
1 + class MataKuliahTeori implements OperasiMataKuliah {
2 +     private String nama;
3 +     public MataKuliahTeori(String nama) { this.nama = nama; }
4 +
5 +     @Override
6 +     public void registrasiKelas() { System.out.println("Registrasi Kuliah Teori: " + nama); }
7 +
8 +     @Override
9 +     public void alokasiAsistenLab() {
10 +         throw new UnsupportedOperationException("Error: Kuliah teori tidak butuh Asisten Lab!");
11 +     }
12 +
13 +     @Override
14 +     public void cekPeralatanPraktikum() {
15 +         throw new UnsupportedOperationException("Error: Kuliah teori tidak punya peralatan praktikum!");
16 +     }
17 + }
```

Penjelasan: Antarmuka OperasiMataKuliah dirancang terlalu gemuk (Fat Interface), sehingga melanggar prinsip ISP karena memaksa kelas konsumen mengimplementasikan metode yang tidak mereka butuhkan. Akibat polusi antarmuka ini, kelas MataKuliahTeori dan Kuliah Lapangan (KKN) dipaksa mengimplementasikan metode alokasiAsistenLab() dan cekPeralatanPraktikum(). Hal ini secara otomatis melanggar LSP. Karena karakteristik aslinya tidak mendukung operasional laboratorium, kelas turunan memanipulasi metode tersebut dengan melemparkan UnsupportedOperationException. Ketika sistem utama melakukan iterasi data secara polimorfis menggunakan tipe induk OperasiMataKuliah, pemanggilan metode lab pada objek MataKuliahTeori akan memicu runtime crash mendadak yang melumpuhkan sistem.

```
TakeHomeTask/src/SistemKRSManger.java
... @@ -0,0 +1,44 @@
1 + class SistemKRSManger {
2 +     private MySQLDatabaseConnection db; // Ketergantungan langsung (Hardcoded)
3 +
4 +     public SistemKRSManger() {
5 +         this.db = new MySQLDatabaseConnection(); // menginstansiasi secara langsung objek MySQLDatabaseConnection
6 +     }
```

Penjelasan: Arsitektur di atas melanggar prinsip DIP karena modul tingkat tinggi (SistemKRSManger) bergantung secara langsung dan kaku (Tight Coupling) pada modul tingkat rendah konkret (MySQLDatabaseConnection). Hubungan dipererat secara paksa melalui instansiasi langsung menggunakan kata kunci new di dalam konstruktor. Kondisi ini membuat tim pengembang mengalami kelumpuhan teknis saat Rektorat meminta migrasi sistem ke infrastruktur Cloud NoSQL. Karena tidak adanya lapisan abstraksi, migrasi database tidak dapat dilakukan seperti sistem plug-and-play, melainkan harus merombak total seluruh deklarasi tipe data dan konstruktor di dalam komponen logika bisnis utama (SistemKRSManger).

Melihat kekurangan arsitektur di atas, tim pengembang SIAKAD memerlukan perubahan baru yang mampu memutus keterikatan kaku antar-komponen sistem. Solusi untuk menyembuhkan seluruh "penyakit" kode tersebut adalah dengan menerapkan SOLID Principles. Melalui pendekatan ini, struktur kode awal yang

monolitik akan direkayasa ulang menjadi komponen-komponen yang modular, memiliki batasan tanggung jawab, serta siap beradaptasi terhadap perubahan regulasi kampus maupun migrasi infrastruktur teknologi di masa depan. Berikut adalah pembahasan transformasi dan rekonstruksi struktur kode SIAKAD setelah mengimplementasikan kelima prinsip SOLID.

```
TakeHomeTask/src/CetakKRSManager.java
... @@ -0,0 +1,16 @@
1 + public class CetakKRSManager {
2 +     public void cetakDraftKRS(String mahasiswa, String matakuliah) {
3 +         System.out.println("Proses Cetak...");
4 +         // Pada semester lalu, seorang program mengubah format kop surat pada file PDF KRS,
5 +         // Simulasi perubahan format string/kop memicu interupsi/bug
6 +         // boolean adaKesalahanFormatKop = true;
7 +         // if (adaKesalahanFormatKop) {
8 +         //     System.out.println("Format Kop Surat Rusak! Proses dihentikan.");
9 +         //     // Akibatnya baris kode penyimpanan di bawah terlewat/gagal dieksekusi
10 +         //     return;
11 +         // }
12 +         // Algoritma penyimpanan data sehingga ribuan data KRS mahasiswa gagal tersimpan ke database.
13 +         // db.hubungkan();
14 +         // db.simpanKRS(mahasiswa, matakuliah);
15 +     }
16 + }
```

```
TakeHomeTask/src/SistemKRSManager.java
... @@ -5,12 +5,6 @@ public SistemKRSManager() {
5 5     this.db = new MySQLDatabaseConnection(); // menginstansiasi secara langsung objek MySQLDatabaseConnection
6 6 }
7 7
8 - // Fungsi Validasi Prasyarat
9 - public boolean validasiPrasyaratMatakuliah(String mahasiswa, String matakuliah) {
10 -     System.out.println("Memvalidasi prasyarat akademik untuk " + mahasiswa);
11 -     return true;
12 - }
13 -
14 8 // If-Else untuk perhitungan UKT berdasarkan Jalur Masuk
15 9 public double hitunguangkuliahTunggal(String jalurMasuk) {
16 10     if (jalurMasuk.equalsIgnoreCase("Reguler")) {
... @@ -26,18 +20,7 @@ public double hitunguangkuliahTunggal(String jalurMasuk) {
26 20     }
27 21 }
28 22
29 - // Fungsi Cetak PDF yang bercampur dengan logika penyimpanan database
30 - public void cetakDraftKRS(String mahasiswa, String matakuliah) {
31 -     System.out.println("Proses Cetak...");
32 -     // Pada semester lalu, seorang program mengubah format kop surat pada file PDF KRS,
33 -     // Simulasi perubahan format string/kop memicu interupsi/bug
34 -     boolean adaKesalahanFormatKop = true;
35 -     if (adaKesalahanFormatKop) {
36 -         System.out.println("Format Kop Surat Rusak! Proses dihentikan.");
37 -         // Akibatnya baris kode penyimpanan di bawah terlewat/gagal dieksekusi
38 -         return;
39 -     }
40 -     // Algoritma penyimpanan data sehingga ribuan data KRS mahasiswa gagal tersimpan ke database.
41 23 + public void simpanKRSkeDatabase(String mahasiswa, String matakuliah) {
42 24     db.hubungkan();
43 25     db.simpanKRS(mahasiswa, matakuliah);
44 26 }
... @@ -0,0 +1,6 @@
1 + public class ValidatorAkademik {
2 +     public boolean validasiPrasyaratMatakuliah(String mahasiswa, String matakuliah) {
3 +         System.out.println("Memvalidasi prasyarat akademik untuk " + mahasiswa);
```

Penjelasan Penerapan Single Responsibility Principle (SRP): Prinsip SRP menetapkan bahwa setiap kelas hanya boleh memikul satu tanggung jawab spesifik di dalam sistem. Berdasarkan hasil audit kode awal, kelas raksasa SistemKRSManager didekonstruksi dengan mengeluarkan fungsi validasi akademik dan fungsi manajemen cetak dokumen ke dalam dua kelas baru yang independen.

1. Kelas ValidatorAkademik: Logika pengecekan kelayakan akademik mahasiswa kini diisolasi sepenuhnya di dalam kelas ValidatorAkademik. Jika di masa mendatang terdapat perubahan regulasi syarat pengambilan SKS, tim pengembang cukup memodifikasi kelas ini tanpa menyentuh modul keuangan maupun modul database.

2. Kelas CetakKRSManger: Tanggung jawab memformat kop surat dan mencetak file PDF dialihkan sepenuhnya ke kelas CetakKRSManger. Melalui pemisahan ini, kode manipulasi string tampilan tidak lagi bercampur dengan logika transaksi data.
3. Kelas SistemKRSManger: Setelah fungsi validasi dan cetak dikeluarkan, kelas SistemKRSManger menjadi jauh lebih ramping. Metode cetakDraftKRS() pada kode awal kini digantikan oleh metode bersih simpanKRSKeDatabase().

```
TakeHomeTask/src/KalkulasiJalurUKT.java
... @@ -0,0 +1,29 @@
1  + interface KalkulasiJalurUKT {
2  +     double hitung();
3  + }
4  +
5  + class UKTRegular implements KalkulasiJalurUKT {
6  +     @Override
7  +     public double hitung() { return 5000000; }
8  + }
9  +
10 + class UKTKaryawan implements KalkulasiJalurUKT {
11 +     @Override
12 +     public double hitung() { return 7500000; }
13 + }
14 +
15 + class UKTInternasional implements KalkulasiJalurUKT {
16 +     @Override
17 +     public double hitung() { return 15000000; }
18 + }
19 +
20 + class UKTBidikmisi implements KalkulasiJalurUKT {
21 +     @Override
22 +     public double hitung() { return 0; }
23 + }
24 +
25 + // Baru
26 + class UKTPEKN implements KalkulasiJalurUKT {
27 +     @Override
28 +     public double hitung() { return 2500000; }
29 + }

TakeHomeTask/src/SistemKRSManger.java
... @@ -5,19 +5,8 @@ public SistemKRSManger() {
5  this.db = new MySQLDatabaseConnection(); // menginstansiasi secara langsung objek MySQLDatabaseConnection
6  }
7
8  - // If-Else untuk perhitungan UKT berdasarkan Jalur Masuk
9  - public double hitungUangKuliahTunggal(String jalurMasuk) {
10 -     if (jalurMasuk.equalsIgnoreCase("Regular")) {
11 -         return 5000000;
12 -     } else if (jalurMasuk.equalsIgnoreCase("Karyawan")) {
13 -         return 7500000;
14 -     } else if (jalurMasuk.equalsIgnoreCase("Internasional")) {
15 -         return 15000000;
16 -     } else if (jalurMasuk.equalsIgnoreCase("Bidikmisi")) {
17 -         return 0;
18 -     } else {
19 -         return 3000000;
20 -     }
21 +
22 + public double kalkulasiUKT(KalkulasiJalurUKT jalurMasuk) {
23 +     return jalurMasuk.hitung();
24 + }
25
26 + public void simpanKRSKeDatabase(String mahasiswa, String matakuliah) {
```

Penjelasan Penerapan Open/Closed Principle (OCP): Prinsip OCP menegaskan bahwa sebuah komponen perangkat lunak harus terbuka untuk diperluas (open for extension), namun tertutup dari modifikasi langsung (closed for modification). Berdasarkan kode awal, penumpukan struktur seleksi If-Else pada metode lama `hitungUangKuliahTunggal()` telah dieliminasi total.

1. Interface KalkulasiJalurUKT: Langkah awal dilakukan dengan membuat sebuah interface kontrak bernama KalkulasiJalurUKT. Setiap skema atau jalur masuk mahasiswa baru kini diwajibkan menjadi objek mandiri yang mengimplementasikan metode `hitung()` secara spesifik.

2. Enkapsulasi: Seluruh isi percabangan logika dari kode awal diurai secara terpisah menjadi kelas konkret masing-masing (UKTReguler, UKTKaryawan, dll). Hal ini agar rumus kalkulasi keuangan tiap jalur terisolasi dengan rapi.
3. KelasKRSManger: Metode lama yang dipenuhi rantai if-else dihapus sepenuhnya. Gantinya, metode baru kalkulasiUKT() kini hanya menerima objek bertipe parameter interface KalkulasiJalurUKT lalu langsung mendelegasikan eksekusinya via jalurMasuk.hitung().
4. Perubahan: Tuntutan dari pihak Rektorat untuk menambahkan skema perhitungan SKS baru untuk program "Merdeka Belajar Kampus Merdeka" (MBKM) telah diakomodasi. Pengembang cukup membuat kelas baru bernama UKTMBKM yang mengimplementasikan interface KalkulasiJalurUKT.

```

TakeHomeTask/src/MatakuliahPraktikum.java
... @@ -1,4 +1,4 @@
1 - class MatakuliahPraktikum implements OperasiMataKuliah {
2 + class MatakuliahPraktikum implements OperasiMataKuliah, OperasiPraktikum {
3     private String nama;
4     public MatakuliahPraktikum(String nama) { this.nama = nama; }

TakeHomeTask/src/MatakuliahTeori.java
... @@ -4,13 +4,4 @@ class MatakuliahTeori implements OperasiMataKuliah {
4
5     @Override
6     public void registrasiKelas() { System.out.println("Registrasi Kuliah Teori: " + nama); }
7
8     @Override
9     public void alokasiAsistenLab() {
10         throw new UnsupportedOperationException("Error: Kuliah teori tidak butuh Asisten Lab!");
11     }
12     @Override
13     public void cekPeralatanPraktikum() {
14         throw new UnsupportedOperationException("Error: Kuliah teori tidak punya peralatan praktikum!");
15     }
16 }

TakeHomeTask/src/OperasiMataKuliah.java
... @@ -1,5 +1,3 @@
1 interface OperasiMataKuliah {
2     void registrasiKelas();
3     void alokasiAsistenLab();
4     void cekPeralatanPraktikum();
5 }

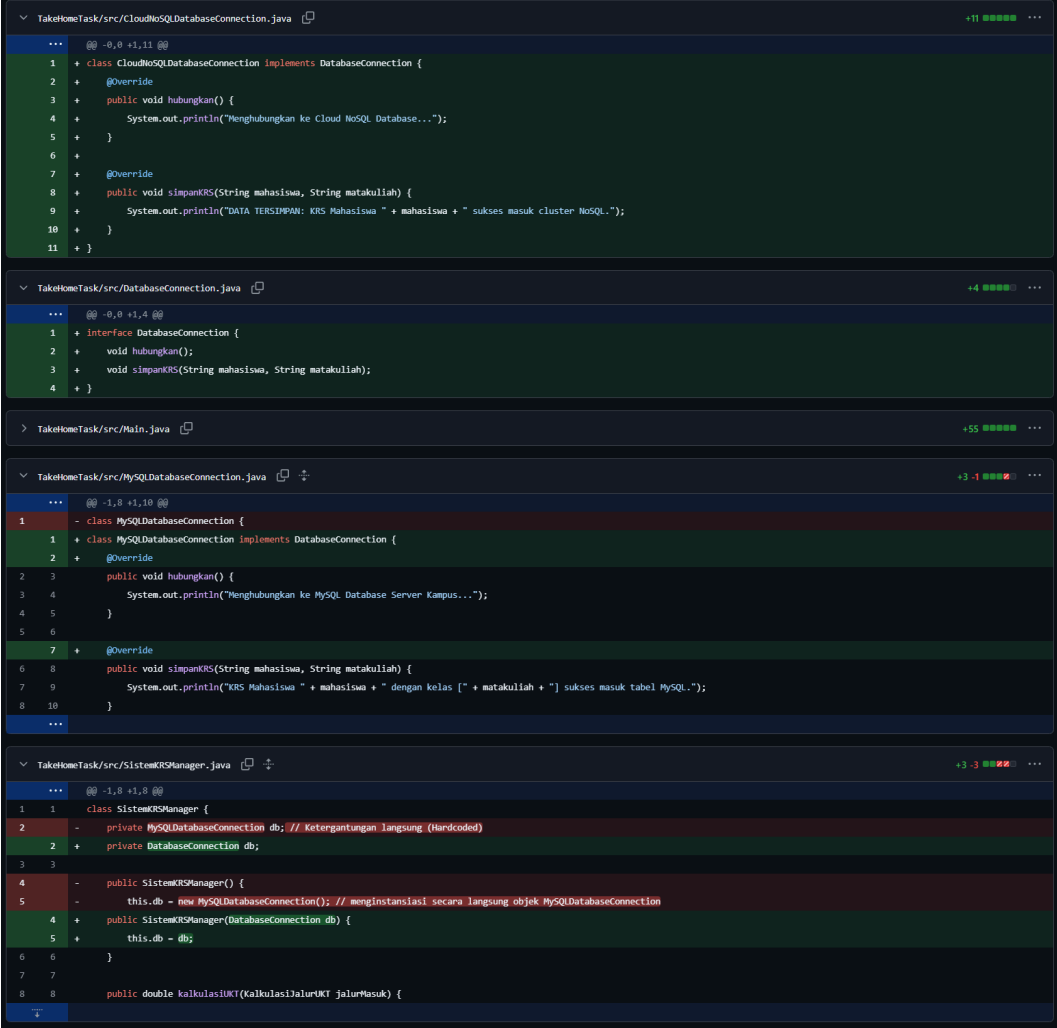
TakeHomeTask/src/OperasiPraktikum.java
... @@ -0,0 +1,5 @@
1 + // Kontrak khusus yang hanya berlaku untuk mata kuliah dengan fasilitas laboratorium
2 + interface OperasiPraktikum {
3 +     void alokasiAsistenLab();
4 +     void cekPeralatanPraktikum();
5 + }

```

Penjelasan Penerapan Liskov Substitution Principle (LSP) & Interface Segregation Principle (ISP): Prinsip ISP menyatakan bahwa sebuah antarmuka (interface) tidak boleh memaksa kelas konsumen untuk mengimplementasikan metode yang tidak digunakannya, sedangkan LSP menjamin bahwa setiap subkelas atau implementasi objek harus dapat menggantikan peran tipe induknya secara transparan tanpa merusak kestabilan program. Berdasarkan kode awal, masalah diperbaiki dengan memotong antarmuka yang besar menjadi kontrak yang spesifik dan terpisah.

1. ISP: Antarmuka induk OperasiMataKuliah dipangkas sehingga hanya menyisakan metode umum registrasiKelas(). Perilaku khusus terkait laboratorium diisolasi ke dalam antarmuka baru bernama OperasiPraktikum.

2. LSP: Metode paksaan alokasiAsistenLab() dan cekPeralatanPraktikum() yang melanggar kontrak dihapus total dari kelas MataKuliahTeori. Kelas ini sekarang murni mengimplementasikan metode registrasiKelas() yang merupakan domain kapabilitas aslinya. Untuk kelas MataKuliahPraktikum, kelas ini menggunakan kapabilitas multiple interface implementation Java dengan mengimplementasikan kedua antarmuka secara bersamaan tanpa melanggar prinsip desain apa pun.



```
*** @ -0,0 +1,11 @@
1 + class CloudNoSQLDatabaseConnection implements DatabaseConnection {
2 +     @Override
3 +     public void hubungkan() {
4 +         System.out.println("Menghubungkan ke Cloud NoSQL Database...");
5 +     }
6 +
7 +     @Override
8 +     public void simpanKRS(String mahasiswa, String matakuliah) {
9 +         System.out.println("DATA TERSIMPAN: KRS Mahasiswa " + mahasiswa + " sukses masuk cluster NoSQL.");
10 +     }
11 + }
```

```
*** @ -0,0 +1,4 @@
1 + interface DatabaseConnection {
2 +     void hubungkan();
3 +     void simpanKRS(String mahasiswa, String matakuliah);
4 + }
```

```
> TakeHomeTask/src/Main.java
+55
```

```
TakeHomeTask/src/MySQLDatabaseConnection.java
+3 -1
```

```
*** @ -1,8 +1,10 @@
1 - class MySQLDatabaseConnection {
2 + class MySQLDatabaseConnection implements DatabaseConnection {
3 +     @Override
4 +     public void hubungkan() {
5 +         System.out.println("Menghubungkan ke MySQL Database Server Kampus...");
6 +     }
7 +
8 +     @Override
9 +     public void simpanKRS(String mahasiswa, String matakuliah) {
10 +         System.out.println("KRS Mahasiswa " + mahasiswa + " dengan kelas [" + matakuliah + "] sukses masuk tabel MySQL.");
11 +     }
12 + }
```

```
TakeHomeTask/src/SistemKRSManager.java
+3 -3
```

```
*** @ -1,8 +1,8 @@
1 class SistemKRSManager {
2 -     private MySQLDatabaseConnection db; // Ketergantungan langsung (Hardcoded)
3 +     private DatabaseConnection db;
4 +
5 +     public SistemKRSManager() {
6 +         this.db = new MySQLDatabaseConnection(); // menginstansiasi secara langsung objek MySQLDatabaseConnection
7 +     }
8 +     public SistemKRSManager(DatabaseConnection db) {
9 +         this.db = db;
10 +     }
11 +
12 +     public double kalkulasiUKT(KalkulasiJalurUKT jalurMasuk) {
```

Penjelasan Penerapan Dependency Inversion Principle (DIP): Prinsip DIP menegaskan bahwa modul tingkat tinggi (logika bisnis utama) tidak boleh bergantung langsung pada modul tingkat rendah (detail teknis/infrastruktur), melainkan keduanya wajib bergantung pada sebuah lapisan abstraksi. Berdasarkan kode, kopling ketat antara kelas pusat bisnis dan database server kampus telah dilepaskan secara total.

1. Interface DatabaseConnection: Langkah dilakukan dengan memperkenalkan sebuah kontrak abstraksi baru berupa interface bernama DatabaseConnection. Antarmuka ini menetapkan fungsi dasar sistem persistensi data tanpa terikat pada spesifikasi vendor database tertentu.

2. Kelas `MySQLDatabaseConnection`: Kelas infrastruktur lama `MySQLDatabaseConnection` dikonfigurasi ulang untuk mengimplementasikan interface `DatabaseConnection`.
3. Perubahan: Tuntutan mendesak dari Rektorat untuk memigrasikan sistem ke infrastruktur Cloud NoSQL (demi menghindari kegagalan sistem saat puncak war KRS) dapat dipenuhi dengan sangat mudah. Tim pengembang cukup membuat kelas baru `CloudNoSQLDatabaseConnection` yang mengimplementasikan interface `DatabaseConnection`.
4. `SistemKRSManger`: Variabel referensi database diubah tipenya dari kelas konkret menjadi tipe abstraksi `DatabaseConnection`. Selain itu, instansiasi kaku `new MySQLDatabaseConnection()` di dalam konstruktor dihapus total dan digantikan dengan mekanisme Constructor Injection (pemasukan objek lewat parameter dari luar).

V. Kesimpulan

Penerapan lima prinsip desain SOLID pada sistem portal akademik SIAKAD terbukti mentransformasikan arsitektur aplikasi menjadi sangat tangkas dan modular. Restrukturisasi berbasis Single Responsibility Principle (SRP) sukses mengisolasi kegagalan operasional, sehingga kesalahan kecil pada aspek presentasi draf PDF tidak akan pernah lagi menginterupsi alur penyimpanan data ke dalam database. Selain itu, penghapusan tumpukan if-else menggunakan Open/Closed Principle (OCP) memberikan fleksibilitas penuh bagi sistem untuk mengakomodasi regulasi baru seperti skema perhitungan SKS MBKM melalui pembuatan ekstensi kelas baru, tanpa risiko merusak rumus perhitungan mahasiswa lama.

Di sisi lain, perbaikan pada hierarki kelas menggunakan Liskov Substitution Principle (LSP) dan Interface Segregation Principle (ISP) dengan memecah antarmuka besar menjadi kontrak yang spesifik berhasil mengeliminasi kemunculan runtime crash akibat `UnsupportedOperationException` secara permanen saat sistem melakukan iterasi data mata kuliah. Implementasi Dependency Inversion Principle (DIP), di mana ketergantungan kaku logika bisnis terhadap database MySQL telah diputus menggunakan lapisan abstraksi dan mekanisme dependency injection. Alhasil, tuntutan migrasi infrastruktur menuju Cloud NoSQL demi mengantisipasi lonjakan akses saat war KRS dapat dieksekusi secara instan dan elegan tanpa perlu membongkar atau memodifikasi satu baris kode pun pada kelas pengelola utama.

VI. Daftar Pustaka

- Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects* (11th ed.). Pearson.
- Schildt, H. (2018). *Java: The Complete Reference* (11th ed.). McGraw-Hill Education.

Tjondrojo, T. F., & Hutagalung, Y. S. C. I. (2026). Modul Praktikum Pemrograman Berorientasi Objek: SOLID Principles. Laboratorium Komputer Program Studi Ilmu Komputer, Universitas Pertamina.